Compact MILP and Column Generation for the T-Adaptive Segment Routing Problem: Implementation and Computational Study on the EURO/ROADEF 2026 SetA Benchmark

Hoàng Tuấn Tài (20261083M) Lê Phạm Thuỷ Tiên (20261075M)
School of Information and Communication Technology
Hanoi University of Science and Technology
Hanoi, Vietnam

***Abstract*—Segment Routing (SR) lets an operator steer traffic away from congested links by inserting a few waypoints into the packet header, while traffic between consecutive waypoints follows the usual shortest paths with Equal-Cost Multi-Path (ECMP) splitting. The T-Adaptive Segment Routing (T-ASR) problem, posed by Orange for the EURO/ROADEF Challenge 2026, asks for segment paths over several time periods in which some links are taken down for maintenance, under a limit on the number of segment changes between consecutive periods. We study the Maximum Link Utilization (MLU) version on the 20 SetA instances with two methods. Method 1 is the compact mixed-integer linear program (MILP) of the challenge, solved with HiGHS; its number of binary variables grows as $|D| \cdot |V|^2$ (up to $9.6 \cdot 10^8$), so it proves optimality on only four instances for one period and three for two periods. Method 2 solves a path formulation by column generation, with certified Lagrangian lower bounds, two-period columns whose budget cost is exact, and MILPs over the generated paths followed by a load-driven neighborhood search for integer solutions. Method 2 returns a feasible solution for all 20 instances within the time limits of the project. On the two-period problem, its best MLU over ten runs equals the best value reported by the 53 teams of the challenge sprint on all 20 instances (in all ten runs on 19 of them), and optimality is proved on 13 instances in every run and on one more in nine runs out of ten (12 instances for one period). On the instances where the compact MILP proves optimality, Method 2, run on the same machine, returns the same values in at most 2.3 s on average, against 9.0 to 2034 s. Every solution in the tables is checked by an independent re-implementation of the routing rules. We also show that comparing path lengths with exact floating-point equality silently corrupts the ECMP coefficients on three instances.**

***Index Terms*—Segment routing, traffic engineering, mixed-integer linear programming, column generation, maintenance planning, ECMP.**

I. Introduction

Orange operates national and international IP/MPLS backbones with thousands of routers and links and tens of thousands of traffic demands. Traffic Engineering (TE) aims to use this infrastructure efficiently, mainly to avoid congestion and to preserve the quality of service. A standard design rule requires the network to remain fully functional when any single link or node is unavailable, including when the operator takes it down on purpose for maintenance [1].

In classical IP routing, packets follow the shortest paths with respect to administrative link weights. This is simple to operate, but many flows tend to share the same few links, and rerouting them requires a new weight setting, which is itself a hard optimization problem [2]. Segment Routing (SR) [3] removes this rigidity: the ingress router writes a short list of waypoints into the packet header, and the packet travels along shortest paths from waypoint to waypoint. Choosing waypoints is therefore enough to move traffic off congested links without touching the weights.

Maintenance makes the problem dynamic. When a link is taken down, shortest paths change and congestion may appear elsewhere, so the waypoints should be adapted. Each change of a segment path is, however, an operational action with a cost and a risk, so the number of changes between two consecutive periods is limited by a budget. The T-Adaptive Segment Routing (T-ASR) problem [1], [4] captures this trade-off. It generalizes the single-period segment routing problem and is NP-hard, inapproximable and parameterized intractable even on very restricted topologies [4], [5].

This paper follows the steps of the course project and goes one step further. Our contributions are:

- an implementation of the MILP formulation of the challenge for one period and for two periods with the linearized budget constraint (Method 1);
- a careful computation of the ECMP split coefficients, a comparison with the path-counting rule of the course material, and an analysis of a floating-point pitfall that silently corrupts the coefficients;
- a path formulation solved by column generation (Method 2), with certified lower bounds, exact handling of the budget, and integer solutions for all 20 instances;
- a computational study on SetA with averages over ten seeded runs wherever the machine allowed them, an independent check of every reported solution, and a comparison with the best values of the challenge.

Section II reviews related work, Section III defines the problem, Section IV gives the MILP, Section V explains the split coefficients, Section VI presents the column generation, Section VII reports the experiments and Section VIII concludes.

II. Related Work

Most optimization work on SR considers a single network state. Bhatia et al. [6] formulate two-segment routing as a linear program. Hartert et al. [7] show that SR reduces congestion and propose a hybrid constraint-programming approach. Jadin et al. [8] handle an arbitrary number of segments with column generation (CG4SR), and Li and Yeung [9] give a MILP formulation for SR-TE. Failure resilience is addressed by Schüller et al. [10], whose MILP minimizes the MLU over failure scenarios, and by the three-segment heuristic of Pereira et al. [11]. Callebaut et al. [12] remove dominated segment paths, in particular paths with loops, which shrinks single-period models considerably.

The multi-period problem with a reconfiguration budget was introduced with the challenge. Benhamiche et al. [4] formulate T-ASR with a lexicographic objective on the sorted vector of link loads and solve it with a sequence of MILPs (ALEXA and STELA) on instances with up to 50 nodes. They also show that, with a budget, an optimal solution may need a segment path with a loop, so the preprocessing of [12] is no longer valid. Jaumard et al. [13] solve the MLU version period by period with column generation on a segment graph and report LP–ILP gaps on SetA, relative to the LP bound, of 3.12% on average at $t = 0$ and 13.05% at $t = 1$. Bouaachra [14] solves the lexicographic version by branch-and-price on *trajectory* columns, tuples of per-period segment paths whose reconfiguration cost is a constant, so that the budget becomes a single linear row. Our Method 2 combines the segment-graph pricing of [8], [13] with such two-period columns for the MLU objective, optimizing both periods jointly.

III. Problem Definition

Network. A network is a directed graph $G = (V, A)$ with $n = |V|$ nodes, a weight $\omega(a) > 0$ used for shortest paths, and a capacity $c(a)$ for every arc $a \in A$.

Forwarding graph and ECMP. The forwarding graph $FG(u, v)$ contains every arc that lies on a shortest u–v path. When $FG(u, v)$ is not a single path, ECMP splits the flow entering a node evenly among the outgoing arcs of that node in $FG(u, v)$. The split coefficient $r(u, v, a, t)$ is the fraction of one unit of flow sent from u to v that crosses arc a in the network of period t.

Segment paths. A segment path $\langle u, w_1, \ldots, w_\ell, v\rangle$ from source u to target v visits the waypoints $w_1, \ldots, w_\ell$ in this order; each pair of consecutive nodes is a segment, routed on its forwarding graph. A path has at most $maxSeg$ segments, including the first and the last one, and uses each segment at most once, as the binary variables of the challenge imply; $\ell = 0$ means plain shortest-path routing.

TABLE I
Parameters and their source in the instance files

Symbol	Meaning	Source
V, A	nodes, arcs	net: nodes, links
$\omega(a)$, $c(a)$	weight, capacity	net: metric, capacity
s_d, t_d	source, target of d	tm: s, t
$\nu(d, t)$	volume of d in period t	tm: v[t]
$maxSeg$	max. segments per path	scenario: max_segments
$q(t)$	arcs down in period t	scenario: interventions
$\kappa(t)$	budget of period t	scenario: budget
$r(i, j, a, t)$	split coefficients	computed (Section V)

Demands and periods. A demand $d = (s_d, t_d) \in D$ has a volume $\nu(d, t)$ in each period $t \in T = \{0, \ldots, h-1\}$. Period 0 is the nominal state; in each $t \in T^* = T \setminus \{0\}$, the arcs $q(t) \subseteq A$ are down and the network is $G_t = (V, A_t)$ with $A_t = A \setminus q(t)$; we set $A_0 = A$.

Budget. Let $\delta(p, ij) = 1$ if segment path p contains segment (i, j). The distance between the routing schemes P and P' is $\mathrm{dist}(P, P') = \sum_d \sum_{i \neq j} |\delta(p_d, ij) - \delta(p'_d, ij)|$, and $\mathrm{dist}(P^{t-1}, P^t) \leq \kappa(t)$ is required for every $t \in T^*$. For example, replacing $\langle u, w, v\rangle$ by $\langle u, v\rangle$ costs 3.

Objective. The load of arc a in period t is $\lambda(a, t) = \frac{1}{c(a)} \sum_d \sum_{i,j} r(i, j, a, t)\, \nu(d, t)\, \delta(p_d, ij)$. The challenge minimizes the vector of all loads in decreasing lexicographic order. As in [13], we minimize its first component, the MLU $\max_{t,a} \lambda(a, t)$; every lexicographically optimal solution is MLU-optimal, but not conversely [4].

IV. MILP Formulation (Method 1)

A. Data

Table I lists the parameters and the JSON fields they are read from. JSON identifiers are mapped to consecutive indices, since they are not always listed in order (setA-14 lists its nodes from 249 down to 0).

B. Variables

A segment path of d is a walk from s_d to t_d that uses each arc at most once in the complete digraph whose arcs are the node pairs (i, j). The binary variable x_{ij}^{dt} equals 1 if the segment path of demand d in period t contains segment (i, j). Variables are created only for pairs with j reachable from i in G_t; the other pairs cannot carry flow. $U \geq 0$ is the MLU and, in the multi-period model, $y_{ij}^{dt} \geq 0$ linearizes the budget.

C. Objective and Constraints

$$\min U$$

$$\text{s.t.} \sum_{j\neq i} x_{ij}^{dt} - \sum_{j\neq i} x_{ji}^{dt} = \begin{cases} 1 & i = s_d \\ -1 & i = t_d \\ 0 & \text{otherwise} \end{cases} \qquad \forall d, i, t \qquad (1)$$

$$\sum_{i\neq j} x_{ij}^{dt} \leq maxSeg \qquad \forall d, t \qquad (2)$$

$$\sum_{d,\, i\neq j} r(i,j,a,t)\, \nu(d,t)\, x_{ij}^{dt} \leq U\, c(a) \qquad \forall t, a \in A_t \qquad (3)$$

$$\sum_{d} \sum_{i\neq j} \left| x_{ij}^{dt} - x_{ij}^{d,t-1} \right| \leq \kappa(t) \qquad \forall t \in T^* \qquad (4)$$

$$x_{ij}^{dt} \in \{0,1\}.$$

Constraint (1) is a flow conservation on the complete digraph of node pairs: for demand d in period t, one more selected segment leaves s_d than enters it, one more enters t_d than leaves it, and every other node (a waypoint, possibly visited several times) has as many selected segments entering as leaving; the selected segments therefore contain a walk from s_d to t_d, which is the segment path (see Section IV-D for the cycles it may also contain). Constraint (2) bounds the number of segments of each path, including the first one (from the source) and the last one (to the target), which models the limited header size. Constraint (3) computes the traffic on each available arc a of period t: every selected segment (i, j) of every demand sends the fraction $r(i, j, a, t)$ of its volume $\nu(d, t)$ through a; dividing by $c(a)$ gives the load $\lambda(a, t)$, and the constraint makes U an upper bound on all loads of all periods, so that minimizing U minimizes the MLU. Constraint (4) counts, for every demand and node pair, whether the segment is used in one period and not in the other, which is $\mathrm{dist}(P^{t-1}, P^t)$ when x contains nothing but the segment paths (Section IV-D), and bounds it by $\kappa(t)$. Since r and ν are data, (3) is linear. The absolute value in (4) is linearized exactly by

$$y_{ij}^{dt} \geq x_{ij}^{dt} - x_{ij}^{d,t-1}, \qquad y_{ij}^{dt} \geq x_{ij}^{d,t-1} - x_{ij}^{dt}, \qquad \textstyle\sum_d \sum_{i\neq j} y_{ij}^{dt} \leq \kappa(t), \qquad (5)$$

with a missing variable (segment unreachable in one of the two periods) taken as 0. The single-period model (Step 5 of the project) is (1)–(3) for $t = 0$; the two-period model (Step 8) adds the second period and (5).

D. Properties

Cost of a change. If the segment set of a demand changes, $x^t - x^{t-1}$ is a non-zero circulation on a digraph without self-loops, so it has at least two non-zero entries. Hence every rerouted demand costs at least 2, and $\kappa(t) = 1$ (setA-05, 10, 17) forbids any change.

Disjoint cycles. Constraint (1) does not forbid x^{dt} from containing, in addition to the s_d–t_d path, a cycle that does not touch it, e.g. $x_{ij}^{dt} = x_{ji}^{dt} = 1$. Such a cycle only adds load, so it never improves the optimal value with one period. With a budget, however, it can make the left-hand side of (4) smaller than the true $\mathrm{dist}(P^{t-1}, P^t)$. Since every T-ASR solution is feasible for the model with the same loads and distance, the two-period model is a relaxation of T-ASR: its lower bounds remain valid, and an optimal solution whose true distance is within the budget is optimal for T-ASR. We therefore read every solution as an Euler walk from s_d (which also handles paths that visit a node twice), discard the remaining cycles, and recompute the loads and dist from the paths.

A valid restriction for one period. Removing a loop from a segment path never increases a load ($r \geq 0$) and never adds a segment. With one period, one may therefore add $\sum_j x_{ji}^{d0} \leq 1$ for $i \neq s_d$ and forbid segments entering s_d or leaving t_d without losing optimality. This restriction is not valid with a budget, since a loop may then be necessary [4].

Size. Each period has about $|D|\, n(n-1)$ binary variables and (3) about $|D| \cdot \mathrm{nnz}(r)$ non-zeros, where $\mathrm{nnz}(r)$ is the number of (segment, arc) pairs with $r > 0$; the two-period model has about three times as many variables (x^0, x^1, y).

V. Computing the Split Coefficients

The coefficients $r(i, j, a, t)$ depend only on G_t and ω, and are needed for every node pair since any node may be a waypoint. For each period we run Dijkstra's algorithm from every node to obtain the distances π_{ij}. Arc $a = (u, v)$ belongs to $FG(i, j)$ if and only if $\pi_{iu} + \omega(a) + \pi_{vj} = \pi_{ij}$. For each target j, the nodes are processed by decreasing $\pi_{\cdot j}$, a topological order of every $FG(\cdot, j)$ since $\omega > 0$. One unit of flow is placed at i, and each node splits the flow it receives evenly among its outgoing arcs (u, v) with $\pi_{uj} = \omega(u, v) + \pi_{vj}$. The flow on arc a is $r(i, j, a, t)$. The coefficients are stored sparsely, as lists of (arc, value) pairs per node pair.

ECMP versus path counting. The course material suggests $r_{ija} = \sigma_{iu}\sigma_{vj}/\sigma_{ij}$, where σ counts shortest paths. This splits the flow evenly over paths, whereas ECMP splits it evenly at each node, and the two differ when shortest paths branch and merge again. On the toy instance of [1], for demand (v_0, v_5), ECMP gives $r(a_{1,3}) = r(a_{1,4}) = 0.5$ and $r(a_{3,5}) = 0.75$, exactly Table 2 of [1], while path counting gives $1/3$, $2/3$ and $2/3$. We use ECMP. On SetA both rules (with the tolerance below) give the same coefficients in both periods, except for one node pair in setA-18 and one in setA-20, neither of which is a demand: at most 1.1% of the node pairs of an instance have several shortest paths.

Floating-point equality. In the four instances with non-integer weights (05, 08, 12, 14; e.g. $\omega = 0.0037$), the same path length summed in two different orders may differ in the last bit. With an exact test, arcs of $FG(i, j)$ are then missed, so the flow is split wrongly and, when a node keeps no outgoing arc, part of the unit of flow sent from i never reaches j, without any error being raised. At $t = 0$ the flow is lost for 12 560 of the 22 350 node pairs of setA-08, 25 144 of 39 800 in setA-12 and 41 280 of 62 250 in setA-14 (12 577, 25 116 and 41 280 at $t = 1$), and to none in setA-05. We compare lengths with a

relative tolerance of 10^{-9}. Over all instances and both periods, equal lengths differ by at most $4.7\cdot10^{-16}$ in relative terms and different lengths by at least $2.7\cdot10^{-6}$, so the tolerance is safe. With the tolerance, the unit of flow reaches j for every pair, and an independent implementation in Python gives identical coefficients on setA-08, 12 and 14.

Validation. For every node pair of every instance and period we checked that the coefficients form a unit flow from i to j, use only arcs of G_t and only arcs of $FG(i,j)$. On the toy instance, the model returns the optimal MLU 0.375 for $t=0$ (0.4375 without waypoints, as in Fig. 4 of [1]) and 0.375 for the two periods. We confirmed both values by enumerating all 206 segment paths with at most $maxSeg = 4$ segments per demand and period.

VI. Column Generation on Segment Paths (Method 2)

The compact model has $|D|\,n(n-1)$ binary variables per period, although a solution uses one segment path per demand. Method 2 therefore takes whole paths as variables and generates them only when they can improve the current solution [8], [13].

A. Path formulation

Let $\mathcal{P}_d$ be the set of segment paths of d with at most $maxSeg$ segments in which no segment is repeated (as in (1)–(4), where x is binary), and let $L_t(p,a) = \sum_{(i,j)\in p} r(i,j,a,t)$ be the flow that one unit sent along p puts on arc a. For one period, with $z_{dp}=1$ if d uses p:

$$\begin{aligned}
&\min\ U\\
&\text{s.t. } \textstyle\sum_{p\in\mathcal{P}_d} z_{dp} = 1 \qquad \forall d && (6)\\
&\textstyle\sum_{d}\sum_{p\in\mathcal{P}_d} \nu(d,0) L_0(p,a) z_{dp} \le U c(a) \quad \forall a && (7)
\end{aligned}$$

with $z\in\{0,1\}$. Setting $z_{dp}=1$ amounts to setting $x^{d0}_{ij}=1$ on the segments of p, so every solution of (6)–(7) is a solution of (1)–(3) with the same U; conversely, the Euler walk from s_d of a solution of (1)–(3) is a path of $\mathcal{P}_d$, and the cycles that (1) also allows only add load. The two formulations therefore have the same optimal value. For two periods, a column is a pair (p_0,p_1) with budget cost $\Delta(p_0,p_1) = |S(p_0)\triangle S(p_1)|$, where $S(p)$ is the segment set of p, and the budget becomes

$$\textstyle\sum_d \sum_{(p_0,p_1)} \Delta(p_0,p_1)\, z_{d,(p_0,p_1)} \le \kappa(1). \qquad (8)$$

For integer z this is exactly $\mathrm{dist}(P^0,P^1)\le\kappa(1)$: no auxiliary variable is needed, and disjoint cycles cannot be used to undercount the distance (Section IV-D).

B. Column generation

The restricted master problem (RMP) is the LP relaxation of (6)–(8) over a subset of columns, initially the no-waypoint path in every period ($\Delta=0$, always feasible). It is re-optimized by the simplex solver of HiGHS (default settings) through its C interface, warm-started from the previous basis when columns are appended. Let μ_d, $w_{a,t}\ge 0$ and $\rho\ge 0$ be the dual variables of (6), (7) (for each period) and (8). The reduced cost of a one-period column (we drop the index $t=0$) is $\nu(d)\sum_a L(p,a)\,w_a - \mu_d$. Because L is a sum over segments, this equals $\nu(d)\sum_{(i,j)\in p} W(i,j) - \mu_d$ with $W(i,j)=\sum_a r(i,j,a)\,w_a \ge 0$. The pricing problem is thus a shortest walk from s_d to t_d with at most $maxSeg$ arcs in the complete digraph with arc lengths W, solved by the Bellman–Ford recursion

$$f_k(v) = \min\Big\{ f_{k-1}(v),\ \min_u \big(f_{k-1}(u) + W(u,v)\big)\Big\}$$

for $k=1,\dots,maxSeg$, with $f_0(s_d)=0$ and $f_0(v)=\infty$ otherwise, in $O(maxSeg\cdot n^2)$ time per source; one recursion serves all demands with the same source. Ties are broken towards fewer segments and an update needs a strict improvement, so, since $W\ge 0$, the walk returned never visits a node twice and its cost is the minimum over all of $\mathcal{P}_d$. For two periods, two candidates are priced for each demand: a path shared by both periods ($\Delta=0$, arc lengths $\nu(d,0)W_0+\nu(d,1)W_1$, one recursion per demand), and the pair of best paths of each period (one recursion per source and period), whose Δ is computed exactly. Columns with a negative reduced cost are added. The loop stops when no candidate has a negative reduced cost, when the lower bound below is within 10^{-6} of the RMP value, or after 600 s. For one period the pricing is exact, so the first condition also proves that the RMP value is the LP bound over all of $\mathcal{P}_d$; for two periods only the second one does.

C. Lower bounds

Lagrangian bound. For any $w\ge 0$ with $\sum_a c(a)w_a > 0$,

$$LB(w) = \frac{\sum_d \nu(d)\min_{p\in\mathcal{P}_d}\sum_a L(p,a)\,w_a}{\sum_a c(a)\,w_a} \qquad (9)$$

is a lower bound on the MLU: if a solution has MLU U, then the flows f_a satisfy $\sum_a w_a f_a \le U\sum_a c(a)w_a$, while $\sum_a w_a f_a = \sum_d \nu(d)\sum_a L(p_d,a)w_a$ is at least the numerator of (9). The minimum is exactly what the pricing computes, so every iteration yields a certified bound; for one period it equals the RMP value at convergence. With two periods, dualizing (8) with ρ adds $\rho\,(\Delta-\kappa)$; two paths with different segment sets differ in at least two segments (Section IV-D), and two paths with the same segment set put the same loads, so the minimum over pairs is at least $\min\{C_d^{\mathrm{sh}},\, C_d^{\mathrm{sep}}+2\rho\}$, with C_d^{sh} the cost of the best shared path and C_d^{sep} that of the best pair; the bound is $\big(\sum_d \min\{C_d^{\mathrm{sh}}, C_d^{\mathrm{sep}}+2\rho\} - \rho\kappa\big)/\sum_{t,a} c(a)w_{a,t}$.

Single-demand bound. The LP lets a demand split over several paths, which is weak when a few demands are large. Demand d alone loads some arc to at least $\min_p \max_a \nu(d,t)L_t(p,a)/c(a)$, and the maximum over d and t is a lower bound on the MLU. It is computed by a bottleneck version of the recursion (maximum instead of sum); taking the maximum over segments of each segment's largest load can only underestimate the load of an arc crossed by two segments, so the computed value is still a valid bound. We report LB as the larger of the two bounds.

D. Integer solutions

MILP over the generated paths. Each demand may use any path that appears in one of its columns. For one period, the MILP has one binary per (demand, path). For two periods, a demand either keeps one path in both periods ($\Delta = 0$) or, if its LP solution uses a column with $\Delta > 0$, takes a pair of different paths with coefficient Δ in the budget row, so the budget is again exact. The constraint $U \geq LB$ is added so that HiGHS stops as soon as it reaches the bound, and the rounded LP solution (largest z per demand, with the budget repaired) is passed as a starting solution. The time limit is 300 s.

Neighborhood search. Generated columns serve the LP, not necessarily the best integer solution; on the toy instance, the MILP over them reaches only 0.4375. We therefore repeat: find the arcs loaded to at least 99.5% of the current MLU; for up to 400 demands crossing them (800 after a round without improvement), compute new paths by a bottleneck recursion that keeps the loads of all other demands fixed and approximately minimizes the largest relative load on the path (ties broken by the total load), plus paths that avoid every segment crossing these arcs; add them to the path sets and re-solve the MILP (limited to 120 s), in which only the demands given new paths and the (at most 1500) most loaded demands crossing arcs loaded to at least 90% of the MLU may move; with two periods, only the former and the 300 most loaded may use two different paths. After a round without improvement the neighborhood is widened (97% and 50% thresholds, up to 2000 most loaded demands). The search stops at the lower bound, after three rounds without improvement (two if no new path was found), or after 780 s (one period) or 2400 s (two periods), so that the total time stays below the limits of the project (1800 s and 3600 s). On the toy instance the first round finds $\langle v_0, v_2, v_5 \rangle$ and the optimum 0.375.

Method 2 thus combines a certified lower bound with a heuristic upper bound; it proves optimality when the two meet. We call a run *Optimal* when the gap $1 - LB/UB$ is at most 10^{-4}, the default relative gap of HiGHS used for Method 1. Every solution is re-evaluated from its paths: loads, MLU per period, number of segments, routability of every segment in G_t, and the true $\mathrm{dist}(P^0, P^1)$.

VII. Computational Experiments

A. Instances

All 20 SetA instances have $h = 2$ periods, $maxSeg = 6$ and exactly one arc down at $t = 1$ (Table II). Every arc has a reverse arc, every source–target pair carries a single demand, and none of the ten instances flagged as multigraphs (05, 06, 08, 10, 12, 13, 14, 16, 17, 19) actually has parallel arcs. Shortest-path routing without waypoints already saturates the most loaded arc in most instances ($\mathrm{MLU}_{\mathrm{SP}} \approx 1$ in Tables III and IV).

B. Environment and protocol

Machine. All runs in the tables were made on one machine: a Google Cloud virtual machine (KVM) with an Intel Xeon CPU at 2.20 GHz, 8 cores (16 logical CPUs), 38.5 MiB of L3 cache and 67.4 GB (62.8 GiB) of RAM, under Ubuntu 24.04.5 LTS (Linux 7.0.0-1014-gcp); GB means 10^9 bytes. The code runs in Docker 29.1.3 containers (Debian 13) with Julia 1.13.2 [15], JuMP 1.32.1 [16], Graphs.jl 1.15.0 and HiGHS 1.15.1 [17] (HiGHS.jl 1.26.1): one container per run for Method 1, and one per step and seed for Method 2, which solves the 20 instances in sequence (plus one per rerun of setA-16 and 19, see below). The machine is shared with other users, so at most seven Method 1 runs, or eight Method 2 processes with two threads each, were executed at a time. For the Method 1 runs of Tables III–V, a run was started only when the free memory covered its estimated peak, and each container had a memory cap. Times are measured under this concurrent load.

Method 1 settings. The time limit is 1800 s for one period (Step 6) and 3600 s for two periods (Step 9). We set `mip_heuristic_effort` to 0.3 instead of the default 0.05, so that HiGHS spends more effort on primal heuristics. In every run of Tables III–V that returns, the branch-and-bound tree has at most one node and, whenever optimality is proved, it is proved at the root. The no-waypoint routing is given as a starting solution; it is always feasible (it uses no budget), so every run has an upper bound. HiGHS reports Optimal when the relative gap is at most 10^{-4} (default `mip_rel_gap`). CPU is the wall-clock time of the whole `optimize!` call, as required by the project: it covers copying the JuMP model into HiGHS and solving it, but not building the model (in the runs that returned, at most 8.4 s on 01–05 and 55 s on 06–09). Each process first builds and solves the toy instance once, so that the measured call does not include Julia's just-in-time compilation. The copy alone, i.e. CPU minus the solve time reported by HiGHS (`solve_time`), takes 1–8 s on 01–03, 30–78 s on 04 and 05, and 372–563 s on 06–09; the mark * in the tables refers to the solve time of HiGHS. Gap is $1 - LB/UB$, with LB the best bound and UB the best solution value. Nodes is the number of branch-and-bound nodes.

Method 1 runs. HiGHS is deterministic, so the ten runs required by the project use `random_seed` $0, \ldots, 9$, each in a separate process. Models with fewer than 10^6 variables (Step 6: 01–05; Step 9: 01–03) are run ten times (the no-loop restriction of Table V once on 04 and 05) and the tables report averages; larger models need about 37–90 minutes and 5–48 GB of RAM per run and are run once, a departure from the ten runs of the project imposed by the shared machine. The peak memory of a run, read from the memory controller of its container, is 3.7–4.0 GB per million variables for the Step 6 models with more than $4 \cdot 10^6$ variables (30.0 GB on setA-07) and 3.1–3.4 GB for the Step 9 models of setA-04, 05, 06, 08 and 09 (measured until they were stopped, so possibly below their peak; see below); smaller models have a larger fixed overhead. At these rates, the models of Step 6 on 10–20 would need from about 59–64 GB (11 and 12) to 3.5–3.8 TB, and those of Step 9 on 07 and 10–20 from at least about 74–82 GB to 9–10 TB, so they are not built: all of them exceed the 67.4 GB of the machine, except Step 6 on 11 and 12, which exceed the memory left free by the other users. Every run is

TABLE II
SETA INSTANCES: NODES, ARCS, DEMANDS, BUDGET AT $t = 1$ AND NUMBER OF BINARY VARIABLES OF THE SINGLE-PERIOD MODEL

Inst	$\|V\|$	$\|A\|$	$\|D\|$	$\kappa(1)$	$\#x$	Inst	$\|V\|$	$\|A\|$	$\|D\|$	$\kappa(1)$	$\#x$	Inst	$\|V\|$	$\|A\|$	$\|D\|$	$\kappa(1)$	$\#x$
01	20	80	40	51	$1.5 \cdot 10^4$	08	150	654	200	13	$4.5 \cdot 10^6$	15	250	1250	600	54	$3.7 \cdot 10^7$
02	30	150	45	63	$3.9 \cdot 10^4$	09	150	750	200	18	$4.5 \cdot 10^6$	16	250	1452	4800	13	$3.0 \cdot 10^8$
03	50	250	20	53	$4.9 \cdot 10^4$	10	150	966	1000	1	$2.2 \cdot 10^7$	17	300	1270	2000	1	$1.8 \cdot 10^8$
04	50	250	200	44	$4.9 \cdot 10^5$	11	200	1000	400	89	$1.6 \cdot 10^7$	18	300	1500	2000	89	$1.8 \cdot 10^8$
05	100	396	100	1	$9.9 \cdot 10^5$	12	200	898	400	13	$1.6 \cdot 10^7$	19	300	1998	6000	13	$5.4 \cdot 10^8$
06	100	500	500	13	$5.0 \cdot 10^6$	13	200	1000	2000	12	$8.0 \cdot 10^7$	20	400	2000	6000	90	$9.6 \cdot 10^8$
07	100	500	800	90	$7.9 \cdot 10^6$	14	250	1108	600	13	$3.7 \cdot 10^7$						

executed under an operating-system timeout of the time limit plus 900 s (2700 s and 4500 s; 5400 s for Step 9 setA-06, the largest model run, with $1.5 \cdot 10^7$ variables, to leave time for building it and copying it into HiGHS), because HiGHS only checks its clock between long steps of presolve and of the root LP and can overrun the limit. A run stopped by this timeout did not prove optimality within the time limit plus 900 s of wall time; its best solution and bound are lost, so Obj is the MLU of the starting solution and the Gap uses the trivial bound $U \geq 0$. Earlier preliminary runs of Method 1, made with the same software, model, time limit and timeout on another machine (AMD EPYC 7713, 32 logical CPUs, 128 GB) under a heavier load (up to about 32 concurrent runs with 8 HiGHS threads each), are not used in the tables; they are only compared with ours in Section VII-F.

Method 2 settings. Time budgets: 600 s for column generation, 300 s for the first MILP and 780 s (Step 6) or 2400 s (Step 9) for the neighborhood search, i.e. nominally 1680 s and 3300 s, below the limits of the project. The column generation is deterministic; the ten runs differ by the `random_seed` of HiGHS in the MILPs $(0, \ldots, 9)$. CPU is the elapsed time of the solving phase (column generation, MILPs including their construction, and search), excluding reading the data and computing r; it includes the compilation of the Julia code for the first instance of a process (a few seconds). Nodes is the total number of branch-and-bound nodes of all MILPs of a run. A first batch used a 600 s budget for the search in both steps. All runs of the two instances whose search can be long (setA-16 in both steps, setA-19 in Step 9) were run or repeated with the final budgets, and the tables use these runs; in every other run the search stopped before 600 s, at the lower bound or after three rounds without improvement, so the budget was not binding. Since the MILPs stop on time limits, a run also depends on the load of the machine and is not exactly reproducible: on setA-19 in Step 9, seeds 2 and 4 missed the optimum in the first batch and reached it in the final runs, and seed 6 did the opposite. Each process used less than 2 GiB of RAM in the samples taken during the runs.

Solution check. After every Method 1 run that returns a solution, the solution is read as Euler walks (Section IV-D); we verify that each path joins s_d to t_d with at most $maxSeg$ segments that are all routable in G_t, recompute the MLU of each period from the paths and compute the true $\mathrm{dist}(P^0, P^1)$. The solutions of both methods in the tables (111 for Method 1, 400 for Method 2) are written in the format of the official checker of the challenge and checked by an independent program written from scratch in Python, which recomputes the shortest paths, the ECMP coefficients and the loads from the JSON files and reproduces the reported MLU.

Interpretation of Step 9. The project text of Step 9 reads "only time period 0", which we take to be copied from Step 6: Step 9 solves the two-period model of Step 8.

C. Method 1, single period (Step 6)

Optimal instances. The single-period model is solved to proven optimality on 01 and 03 in every run, on 02 in 9 of the 10 runs and on 05 in 2; all runs that prove optimality give the same value. Segment Routing lowers the MLU by 7.1% (setA-01), 45.1% (02), 5.6% (03) and 65.3% (05) with respect to shortest-path routing. The CPU time varies strongly with the seed (setA-02: 100–1803 s, median 305 s; seed 7 stops at the time limit without improving the starting solution) and is not monotone in the model size: setA-02, with $3.9 \cdot 10^4$ variables, takes longer than setA-03 with $4.9 \cdot 10^4$ (median 51 s). On setA-05, three runs return on the time limit, after 1872 to 2298 s, with the optimal value as bound but an MLU of 0.147, 0.164 and 0.195 instead of 0.1155, and in five runs HiGHS overruns its limit and the process is stopped at 2700 s: with up to seven runs sharing the eight cores, whether optimality is proved within 1800 s depends on the seed and on the load.

Root bound. In every run that returns, the branch-and-bound tree has at most one node, so the bound obtained at the root (LP relaxation plus HiGHS cuts) is already the optimal value whenever optimality is proved. The time is spent solving the root LP and searching for an integer solution that reaches the bound, which motivated the larger `mip_heuristic_effort`.

Unsolved instances. On setA-04 (200 demands), every run stops at the time limit with the same root bound $LB = 0.4591$, while nine of the ten seeds never improve the starting solution (MLU 0.6918) and only seed 2 finds a better one (0.5467). The difficulty is thus to find good integer solutions, not to prove the bound. On the models with 4.5–$7.9 \cdot 10^6$ variables (06–09), HiGHS does not finish within 1800 s: on setA-06 and 07 it does not finish the root LP, so there is no bound and the best solution is the starting one; on setA-08 it reduces the MLU from 1.0 to 0.5832 with a bound of 0.3189 (gap 45.3%); on setA-09 it keeps the starting solution (0.6111) with

TABLE III
METHOD 1, STEP 6: PERIOD $t = 0$ ONLY, TIME LIMIT 1800 S. $\mathrm{MLU_{SP}}$: MLU OF SHORTEST-PATH ROUTING (NO WAYPOINTS)

Inst	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert D\rvert$	$\mathrm{MLU_{SP}}$	Obj	CPU (s)	Status	Gap	Nodes	Runs
01	20	80	40	1.0000	0.9294	9.0	Optimal	0	1.0	10
02	30	150	45	1.0000	0.5486	545.6	Opt. 9/10	4.51%	0.7	10
03	50	250	20	1.0000	0.9435	91.7	Optimal	0	1.0	10
04	50	250	200	0.6918	0.5467	1847*	Not Opt.	31.88%	0.0	10
05	100	396	100	0.3333	0.1155	1736*†	Opt. 2/10	59.15%	0.4	10
06	100	500	500	1.0000	1.0000	2192	Not Opt.	100%	0.0	1
07	100	500	800	0.9080	0.9080	2373	Not Opt.	100%	0.0	1
08	150	654	200	1.0000	0.5832	2219	Not Opt.	45.31%	0.0	1
09	150	750	200	0.6111	0.6111	2176	Not Opt.	17.30%	0.0	1
10	150	966	1000	1.0000	1.0000	–	Not Opt.§	100%	–	0
11	200	1000	400	1.0000	1.0000	–	Not Opt.§	100%	–	0
12	200	898	400	1.0001	1.0001	–	Not Opt.§	100%	–	0
13	200	1000	2000	1.0000	1.0000	–	Not Opt.§	100%	–	0
14	250	1108	600	1.0000	1.0000	–	Not Opt.§	100%	–	0
15	250	1250	600	1.0000	1.0000	–	Not Opt.§	100%	–	0
16	250	1452	4800	0.9672	0.9672	–	Not Opt.§	100%	–	0
17	300	1270	2000	1.0001	1.0001	–	Not Opt.§	100%	–	0
18	300	1500	2000	0.6334	0.6334	–	Not Opt.§	100%	–	0
19	300	1998	6000	1.0000	1.0000	–	Not Opt.§	100%	–	0
20	400	2000	6000	1.0000	1.0000	–	Not Opt.§	100%	–	0

Obj: best value over the runs. CPU, Gap and Nodes: averages over the runs (Gap 0 for an optimal run; a run stopped by the hard stop counts with Gap 100% and is left out of CPU and Nodes). Opt. k/n: optimal in k of the n runs; Not Opt.: no run proves optimality. CPU: wall time of the `optimize!` call (copy into HiGHS and solve), without model building (Section VII-B). *: the solve time of HiGHS itself exceeded the time limit by more than 2% in at least one run. †: process stopped by the hard stop (time limit + 900 s) before HiGHS returned, in every run (after the Status) or in some runs (after the CPU); the Obj of such a run is the MLU of the starting solution and its Gap uses the trivial bound 0. §: model not built for lack of memory (Section VII-B); no run returned, so the only solution of Method 1 is the starting one (Obj $= \mathrm{MLU_{SP}}$) and Gap uses the trivial bound 0. "–": not available.

a bound of 0.5054 (gap 17.3%). Instances 10–20 are not built (Section VII-B); they are reported as Not Optimal with the starting solution as their only solution.

No-loop restriction. Table V compares (1)–(3) with the restriction of Section IV-D. It keeps the optimal value on 01, 02, 03 and 05 and is faster on 01–03 (median time 2.4, 13 and 1.5 times smaller). On setA-04 it finds a much better solution (0.4791 instead of 0.5467 at best) with the same root bound, and its single run on setA-05 proves optimality in 1271 s, whereas the original model does so in two runs out of ten. On 04 and 05 the comparison rests on a single run of the restriction, so it is only indicative. Since the restriction is not valid with a budget, Tables III and IV use the original model.

D. Method 1, two periods with budget (Step 9)

Solved instances. The two-period model is solved to optimality on 01 and 02 in every run, and on 03 in 7 of the 10 runs, always with the same value. On 01 and 03, the two-period MLU equals the optimum of period 0 alone (0.9294 and 0.9435), while period 1 solved alone, without budget, reaches 0.3464 and 0.4026: period 0 is the bottleneck. On setA-02, shortest-path routing overloads one arc to 109.8% at $t = 1$; the model brings the MLU down to 0.9031, exactly the optimum of period 1 solved alone, so the limit is the network without its missing arc under the $t = 1$ traffic, not the budget $\kappa(1) = 63$. Because $\min U$ only looks at the most loaded period, the setA-02 solutions have an MLU of 0.80–0.88 at $t = 0$, far above the 0.5486 reachable when $t = 0$ is optimized alone; the lexicographic objective of the challenge would reduce these loads. On setA-03, seed 2 finds the optimal value 0.9435 but does not prove it within 3600 s (lower bound 0.73), seed 1 stops at 0.9822 and seed 7 does not improve the starting solution (MLU 1.0), with lower bounds of 0.67–0.68, while the seven other seeds prove optimality in 12–43 minutes.

Unsolved instances. On setA-04 ($1.5 \cdot 10^6$ variables) and setA-05 ($3.0 \cdot 10^6$; with $\kappa(1) = 1$ no demand may change its path), no bound is returned and the processes are stopped at 4500 s of wall time, after using 5.1 and 9.9 GB of RAM. setA-06 ($1.5 \cdot 10^7$ variables), 08 and 09 ($1.3 \cdot 10^7$ each) used 47.5, 43.7 and 42.0 GB and were stopped at 5400, 4500 and 4500 s, also without a bound. setA-07 and 10–20 are not built.

Solution check. The check of Section VII-B accepts all 114 solutions returned by Method 1 (Tables III–V and the runs of period 1 alone): every recovered path joins its source to its target with at most $maxSeg$ segments, and the MLU recomputed from the paths equals the objective value; the independent Python check accepts the 111 of them that belong to the tables. Many solutions contain disjoint cycles (26 of the 49 solutions of Table III and 20 of the 30 of Table IV, with up to 30 and 75 segments); they are an artifact of the formulation and are discarded when the paths are read. In Step 9 the true $\mathrm{dist}(P^0, P^1)$ is within the budget: at most 25 on setA-01 ($\kappa(1) = 51$), 5 on setA-02 and 4 on setA-03. Different seeds return different optimal routings, some keeping every path unchanged between the periods; every optimal value in Table IV is attained by a valid T-ASR solution.

TABLE IV

METHOD 1, STEP 9: PERIODS $t \in \{0, 1\}$ WITH THE BUDGET CONSTRAINT, TIME LIMIT 3600 S. OBJ IS THE MLU OVER BOTH PERIODS; $\mathrm{MLU_{SP}}$: THE LARGER NO-WAYPOINT MLU OF THE TWO PERIODS

Inst	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert D\rvert$	$\mathrm{MLU_{SP}}$	Obj	CPU (s)	Status	Gap	Nodes	Runs
01	20	80	40	1.0000	0.9294	148.1	Optimal	0	1.0	10
02	30	150	45	1.0981	0.9031	236.5	Optimal	0	1.0	10
03	50	250	20	1.0000	0.9435	2034	Opt. 7/10	8.69%	0.7	10
04	50	250	200	1.0000	1.0000	>4500	Not Opt.†	100%	–	1
05	100	396	100	1.0000	1.0000	>4500	Not Opt.†	100%	–	1
06	100	500	500	1.0000	1.0000	>5400	Not Opt.†	100%	–	1
07	100	500	800	1.0000	1.0000	–	Not Opt.§	100%	–	0
08	150	654	200	1.0000	1.0000	>4500	Not Opt.†	100%	–	1
09	150	750	200	1.0000	1.0000	>4500	Not Opt.†	100%	–	1
10	150	966	1000	1.0000	1.0000	–	Not Opt.§	100%	–	0
11	200	1000	400	1.0000	1.0000	–	Not Opt.§	100%	–	0
12	200	898	400	1.0001	1.0001	–	Not Opt.§	100%	–	0
13	200	1000	2000	1.0000	1.0000	–	Not Opt.§	100%	–	0
14	250	1108	600	1.0000	1.0000	–	Not Opt.§	100%	–	0
15	250	1250	600	1.0000	1.0000	–	Not Opt.§	100%	–	0
16	250	1452	4800	1.0000	1.0000	–	Not Opt.§	100%	–	0
17	300	1270	2000	1.0001	1.0001	–	Not Opt.§	100%	–	0
18	300	1500	2000	1.0000	1.0000	–	Not Opt.§	100%	–	0
19	300	1998	6000	1.0000	1.0000	–	Not Opt.§	100%	–	0
20	400	2000	6000	1.0000	1.0000	–	Not Opt.§	100%	–	0

Obj: best value over the runs. CPU, Gap and Nodes: averages over the runs (Gap 0 for an optimal run; a run stopped by the hard stop counts with Gap 100% and is left out of CPU and Nodes). Opt. k/n: optimal in k of the n runs; Not Opt.: no run proves optimality. CPU: wall time of the `optimize!` call (copy into HiGHS and solve), without model building (Section VII-B). †: process stopped by the hard stop (time limit + 900 s; 5400 s for setA-06) before HiGHS returned, in every run (after the Status) or in some runs (after the CPU); the Obj of such a run is the MLU of the starting solution, its Gap uses the trivial bound 0, and a CPU of the form $>t$ is the wall time of the whole process (start-up, building, copy and solve) when it was stopped. §: model not built for lack of memory (Section VII-B); no run returned, so the only solution of Method 1 is the starting one (Obj $= \mathrm{MLU_{SP}}$) and Gap uses the trivial bound 0. "–": not available.

TABLE V

METHOD 1, STEP 6 ON 01–05: MODEL (1)–(3) VERSUS THE NO-LOOP RESTRICTION (ONE RUN OF THE LATTER ON 04 AND 05). OBJ: BEST VALUE; CPU AND GAP: AVERAGES; SYMBOLS AS IN TABLE III

	Model (1)–(3)				With no-loop constraints			
Inst	Obj	CPU	Gap	Runs	Obj	CPU	Gap	Runs
01	0.9294	9.0	0	10	0.9294	1.9	0	10
02	0.5486	545.6	4.51%	10	0.5486	28.5	0	10
03	0.9435	91.7	0	10	0.9435	31.0	0	10
04	0.5467	1847*	31.88%	10	0.4791	2090*	4.18%	1
05	0.1155	1736*†	59.15%	10	0.1155	1271	0	1

E. Method 2 (column generation)

Table VI reports Method 2 on all 20 instances over ten runs. The ten runs return the same objective value on every instance and in both steps, with one exception: on setA-19 in Step 9, nine runs reach the optimum 0.046838 and one (seed 6) stops at 0.048712, for the reason discussed in Section VII-F.

Coverage and time. Every run returns a solution within the time limits: the longest run takes 1097 s in Step 6 and 1225 s in Step 9 (setA-16). In every run, column generation closes, i.e. the Lagrangian bound reaches the value of the RMP, within 67 iterations and 130 s, and the RMP ends with between 25 and 23 281 columns (including the $|D|$ initial ones), against up to $9.6 \cdot 10^8$ variables per period for Method 1. On 16 of the 20 instances, every run takes less than 25 s in both steps.

Quality. In Step 9, the best MLU of Method 2 equals the best MLU reported by the 53 teams of the challenge sprint [18] on all 20 instances, and every run reaches it on 19 of them: truncated to six decimals, as printed by the official checker, the two values coincide. Since the reference values come from the official checker, this agreement indicates that our split coefficients and loads match those of the challenge, at least on the arcs that set the MLU. Optimality is proved in every run on 12 instances in Step 6 and on 13 in Step 9, and in nine of ten runs on setA-19 in Step 9. Where it is not, the average gap is below 1% on 04, 13 and 16 (both steps) and between 1.8% and 9.6% on 10, 11, 12 and 14; on 03 it is 32.8% in both steps. On setA-03 at least, the gap comes from the lower bound rather than from the solution: Method 1 proves that 0.9435 is optimal in both steps, while the bound of Method 2 is only 0.6337. setA-03 has 20 demands, some of them large, so the LP, which may split a demand over several paths, is weak. On 07 and 18 in Step 6, and on 18 in Step 9, the proved optimum equals the no-waypoint MLU: the most loaded arc cannot be avoided.

Reduction of the MLU. With respect to shortest-path routing, Method 2 lowers the MLU by 40.6% (Step 6) and 31.6% (Step 9) in median over the 20 instances, and by up to 96.1% (setA-13, Step 6). Over the 20 instances, the mean MLU of Method 2 (averaged over its runs) is 0.481 in Step 6 and 0.572 in Step 9, against 0.839 and 0.989 for the best values of Method 1 in Tables III and IV, where an instance without a solution keeps the no-waypoint MLU.

F. Comparison of the two methods

Consistency. On every instance where Method 1 proves optimality (01, 02, 03, 05 in Step 6; 01, 02, 03 in Step 9),

TABLE VI

METHOD 2 (COLUMN GENERATION): AVERAGES OVER TEN RUNS (RANDOM_SEED $0, \ldots, 9$) FOR EVERY INSTANCE AND STEP. OBJ: BEST MLU OVER THE RUNS (THE TEN RUNS AGREE EXCEPT ON SETA-19 IN STEP 9, WHERE ONE RUN ENDS AT 0.0487); CPU, GAP AND NODES: AVERAGES OVER THE RUNS, WITH GAP 0 FOR AN OPTIMAL RUN; LB: CERTIFIED LOWER BOUND; OPT.: GAP $1 - LB/UB \leq 10^{-4}$; OPT. k/n: OPTIMAL IN k OF THE n RUNS. BEST: BEST MLU OF THE CHALLENGE SPRINT (53 TEAMS, TWO PERIODS) [18]

	Step 6 (period 0, limit 1800 s)						Step 9 (periods 0 and 1 with budget, limit 3600 s)						
Inst	Obj	LB	CPU (s)	Status	Gap	Nodes	Obj	LB	CPU (s)	Status	Gap	Nodes	Best
01	0.9294	0.9294	0.0	Opt.	0	0	0.9294	0.9294	2.3	Opt.	0	0	0.9294
02	0.5486	0.5486	0.1	Opt.	0	3	0.9031	0.9031	0.0	Opt.	0	1	0.9031
03	0.9435	0.6337	0.1	Not Opt.	32.84%	4	0.9435	0.6337	1.1	Not Opt.	32.84%	4	0.9435
04	0.4591	0.4591	0.5	Not Opt.	0.011%	265	0.5812	0.5811	4.8	Not Opt.	0.016%	9102	0.5812
05	0.1155	0.1155	0.0	Opt.	0	0	0.2050	0.2050	0.0	Opt.	0	0	0.2050
06	0.0986	0.0986	0.6	Opt.	0	17	0.0986	0.0986	3.1	Opt.	0	7	0.0986
07	0.9080	0.9080	0.1	Opt.	0	0	0.9080	0.9080	0.1	Opt.	0	0	0.9080
08	0.3189	0.3189	0.1	Opt.	0	2	0.3189	0.3189	0.3	Opt.	0	3	0.3189
09	0.5054	0.5054	0.1	Opt.	0	1	0.8497	0.8497	0.1	Opt.	0	0	0.8497
10	0.0696	0.0683	7.8	Not Opt.	1.79%	393	0.0717	0.0683	22.6	Not Opt.	4.76%	488	0.0717
11	0.6446	0.6131	0.5	Not Opt.	4.88%	10	0.7858	0.7858	0.2	Opt.	0	0	0.7858
12	0.6401	0.5789	0.2	Not Opt.	9.55%	3	0.8799	0.8799	0.2	Opt.	0	0	0.8799
13	0.0389	0.0389	28.2	Not Opt.	0.100%	8917	0.0410	0.0408	367	Not Opt.	0.57%	11782	0.0410
14	0.4342	0.4239	0.4	Not Opt.	2.37%	4	0.5176	0.5074	1.0	Not Opt.	1.97%	22	0.5176
15	0.8252	0.8252	0.2	Opt.	0	0	0.8987	0.8987	0.4	Opt.	0	0	0.8987
16	0.0389	0.0386	841	Not Opt.	0.88%	64662	0.0443	0.0441	1139	Not Opt.	0.46%	9755	0.0443
17	0.4242	0.4242	0.5	Opt.	0	0	0.4242	0.4242	0.9	Opt.	0	0	0.4242
18	0.6334	0.6334	0.4	Opt.	0	0	1.0000	1.0000	0.8	Opt.	0	0	1.0000
19	0.0468	0.0468	409	Opt.	0	4921	0.0468	0.0468	861	Opt. 9/10	0.38%	4004	0.0468
20	0.9913	0.9913	4.3	Opt.	0	0	0.9913	0.9913	6.2	Opt.	0	0	0.9913

Method 2 returns the same value, and every Method 1 solution is at least the lower bound of Method 2. The bounds also agree: on setA-04 in Step 6, the root bound of Method 1 and the lower bound of Method 2 coincide (0.4591), and Method 2 finds 0.4591 where Method 1 reaches at best 0.5467 (0.4791 with the no-loop restriction); on setA-08 and 09 in Step 6, the bounds of Method 1 (0.3189 and 0.5054) equal the values that Method 2 proves optimal. The two methods share only the data reader and the computation of r (checked separately by the Python program), so this agreement cross-checks both optimization layers.

Time. Both methods ran on the same machine. On the instances where Method 1 proves optimality, Method 2 returns the same values with an average time of at most 2.3 s (mostly compiling the Julia code on setA-01 in Step 9), against 9.0 to 2034 s for Method 1 (Tables III, IV and VI). Both measures cover passing the model to the solver; that of Method 1 leaves out building the JuMP model (at most 8.4 s on these instances), whereas that of Method 2 includes building its MILPs; these protocol differences, like the different concurrent loads, are small compared with the gap between the methods.

More memory would not remove the limits of Method 1. They come from the compact model. Its memory grows with $|D| \cdot n^2$ (3.1–4.0 GB per million variables on our largest models), so the largest models would need about 3.5–3.8 TB for one period and 9–10 TB for two; twice the memory of our machine would only add the one-period models of 10–12 and the two-period model of 07. In every run of Tables III–V that returns, the branch-and-bound tree has at most one node, so the time goes into the root LP and the primal heuristics on a single model, and more cores mainly allow more runs in parallel. Whether more cores would help cannot be settled from our data: the preliminary runs on a machine with twice as many logical CPUs and about twice the memory had less CPU per run (Section VII-B), yet gave the same best values on every instance run on both machines, no root bound on 06 and 07 in Step 6 either, and no result on any two-period model from setA-04 on; in Step 6, setA-02 and 05 were optimal in 10 and 6 of 10 runs there (9 and 2 here), and setA-09 was stopped by the timeout there but returned a bound here. Method 2 avoids the $|D| \cdot n^2$ model altogether: it works with at most 23 281 columns, each pricing round is a set of shortest-path recursions in $O(maxSeg \cdot n^2)$, and the MILPs it solves contain only the generated paths.

Limits of Method 2. Method 1 is the model prescribed by the project, and its root bound, thanks to the presolve and cuts of HiGHS, can be much stronger than the path LP of Method 2 (the LP relaxation of the compact model alone is not stronger, since every path solution is one of its solutions): on setA-03 it proves the optimum 0.9435, where Method 2 stops at a bound of 0.6337. Method 2 is a heuristic for the upper bound and generates only paths that visit no node twice, whereas with a budget an optimum may need a loop (Section IV-D); its lower bounds cover all paths, but its proofs of optimality depend on the lower bound reaching the solution value, and its search may stop early. In the run of setA-19 (Step 9, seed 6) that ends at 0.048712, the MILPs of the last three rounds of the search all stop on their 120 s limit without improvement, so the search ends after three non-improving rounds while about 1600 s of its 2400 s budget remain; the nine other runs, with the same configuration, reach the optimum 0.046838, as did seed 6 in the first batch. A round whose MILP stops on its time limit is inconclusive rather than unsuccessful; counting only proved non-improving rounds, and spending the remaining budget, is

a natural refinement.

VIII. Conclusion

We implemented the MILP model of the T-ASR challenge with the MLU objective, for one period and for two periods with the linearized budget constraint, and evaluated it on the SetA benchmark with checks of every solution. For one period the formulation is exact; with two periods it may undercount dist through disjoint cycles, which is why every solution was checked. The compact model proves optimality on the small instances and reduces the MLU by up to 65.3% with respect to shortest-path routing, but its size, $|D|\cdot|V|^2$ binary variables, limits it: on our shared machine with 67.4 GB of RAM, the model was not built, for lack of available memory, for 11 of the 20 instances with one period and for 12 with two periods, and no bound is obtained within the time limit from about $5 \cdot 10^6$ variables for one period and $1.5 \cdot 10^6$ for two.

A path formulation solved by column generation removes this limit. Its pricing problem is a shortest path with at most $maxSeg$ arcs on the segment graph, every iteration yields a certified lower bound, pairs of paths make the budget exact for two periods, and MILPs over the generated paths combined with a load-driven neighborhood search produce integer solutions. It returns a solution for all 20 instances in at most 1225 s; on the two-period problem its best MLU over ten runs matches the best MLU of the challenge sprint on all 20 instances, and optimality is proved on 14 of them (in nine of ten runs on one). Three practical lessons emerge: path lengths must be compared with a tolerance when weights are not integers; solutions of the compact model must be read as Euler walks and checked for disjoint cycles when a budget is present; and the MLU objective does not improve the periods that are not the bottleneck.

Natural extensions are a stronger lower bound when a few demands are large (branch-and-price on the path variables, as in [14]), the preprocessing of dominated segment paths [12], and the sequential MILPs of [4] over the generated paths for the full lexicographic objective of the challenge.

Acknowledgment

We thank Orange and the EURO/ROADEF Challenge 2026 organizers for the problem statement, the SetA instances, the checker and the published sprint results.

References

[1] A. Benhamiche, Y. Carlinet, M. Chopin, E. Gourdin, and N. Perrot, "Challenge EURO/ROADEF 2026: Problem description 'Keep The Flow!'," Orange Research, 2026.

[2] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in *Proc. IEEE INFOCOM*, vol. 2, 2000, pp. 519–528.

[3] C. Filsfils, Ed., S. Previdi, Ed., L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir, "Segment routing architecture," RFC 8402, Jul. 2018.

[4] A. Benhamiche, K. Bouaachra, Y. Carlinet, M. Chopin, E. Gourdin, and N. Perrot, "On the T-adaptive segment routing problem," arXiv:2609.32896, 2026.

[5] C. Bazgan, M. Chopin, A. Nichterlein, and C. Richer, "Parameterized complexity of segment routing," in *Proc. IEEE INFOCOM*, 2025, pp. 1–10.

[6] R. Bhatia, F. Hao, M. Kodialam, and T. V. Lakshman, "Optimized network traffic engineering using segment routing," in *Proc. IEEE INFOCOM*, 2015, pp. 657–665.

[7] R. Hartert, P. Schaus, S. Vissicchio, and O. Bonaventure, "Solving segment routing problems with hybrid constraint programming techniques," in *Proc. CP*, LNCS 9255, 2015, pp. 592–608.

[8] M. Jadin, F. Aubry, P. Schaus, and O. Bonaventure, "CG4SR: Near optimal traffic engineering for segment routing with column generation," in *Proc. IEEE INFOCOM*, 2019, pp. 1333–1341.

[9] X. Li and K. L. Yeung, "Traffic engineering in segment routing networks using MILP," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 3, pp. 1941–1953, 2020.

[10] T. Schüller, N. Aschenbruck, M. Chimani, and M. Horneffer, "Failure resiliency with only a few tunnels – enabling segment routing for traffic engineering," *IEEE/ACM Trans. Netw.*, vol. 29, no. 1, pp. 262–274, 2021.

[11] V. Pereira, M. Rocha, and P. Sousa, "Traffic engineering with three-segments routing," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 3, pp. 1896–1909, 2020.

[12] H. Callebaut, J. De Boeck, and B. Fortz, "Preprocessing for segment routing optimization," *Networks*, vol. 82, no. 4, pp. 459–478, 2023.

[13] B. Jaumard, N. P. Tran, and J. M. Ziazet, "Segment routing traffic engineering with time-based reconfiguration constraints," arXiv:2610.04759, 2026.

[14] K. Bouaachra, "Lexicographic minimax load balancing for T-adaptive segment routing," arXiv:2609.36338, 2026.

[15] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Rev.*, vol. 59, no. 1, pp. 65–98, 2017.

[16] M. Lubin, O. Dowson, J. Dias Garcia, J. Huchette, B. Legat, and J. P. Vielma, "JuMP 1.0: Recent improvements to a modeling language for mathematical optimization," *Math. Program. Comput.*, vol. 15, no. 3, pp. 581–589, 2023.

[17] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Math. Program. Comput.*, vol. 10, no. 1, pp. 119–142, 2018.

[18] Orange Open Source, "EURO/ROADEF Challenge 2026: instances, checker and sprint results," https://gitlab.com/Orange-OpenSource/network-optimization-tools/challenge-roadef-2026, 2026.